Open in Colab

`import` Statements

`import` statements are uset to import non builtin functions and objects from other python files.

To use a module which is already in standart library of python (the modules installed when installing Python) use `import` keyword like:

Some Modules in Standart Library

Regular Expressions

In [1]:
```
import re
```

Now you can use `re` to access the functions, classes and objects inside `re` module.

To search a regular expression use `re.search` function.

In [2]:
```
re.search(r"(\d+)","This function will match only: 1234").groups()
```

Out[2]:
```
('1234',)
```

Here `()` inside the regex used to group search results.

To search a regular expression for exact match use `re.match` instead.

In [3]:
```
re.match(r".*(\d+)","This function will match only: 1234").groups()
```

Out[3]:
```
('4',)
```

To find all non-overlapping string with a regular expression, use:

In [4]:
```
re.findall(r"\d+","This function will match only: 1234")
```

Out[4]:
```
['1234']
```

You can use regular expressions to substitude (replace) a text as well.

In [5]:
```
re.sub(r"\d","_","This function will replace the numbers with underscores: 1234")
```

Out[5]:
```
'This function will replace the numbers with underscores: ____'
```

In [6]:
```
re.sub(r"(\d)",r"!\1","This function will seperate the numbers with !: 1234")
```

Out[6]:
```
'This function will seperate the numbers with !: !1!2!3!4'
```

Accessing Web Resources

Some modules have submodules. To import them seperate them with: `.`

In [7]:
```
import urllib.request
```

Now you can use `urllib.request` to access the functions, classes and objects in `request` submodule of `urllib` module.

`urllib.request.urlopen()` is used to open web pages for reading. Just like files.

In [8]:
```
with urllib.request.urlopen("https://example.com/") as file:
  data = file.read()
```

After finishing the `with` block the connection to the web page is closed.

To view the web page first the data shoud be converted to string using `.decode()`

In [9]:
```
data.decode()
```

Out[9]:
```
'<!doctype html><html lang="en"><head><title>Example Domain</title><meta name="viewpo
rt" content="width=device-width, initial-scale=1"><style>body{background:#eee;width:6
0vw;margin:15vh auto;font-family:system-ui,sans-serif}h1{font-size:1.5em}div{opacity:
0.8}a:link,a:visited{color:#348}</style><body><div><h1>Example Domain</h1><p>This dom
ain is for use in documentation examples without needing permission. Avoid use in ope
rations.<p><a href="https://iana.org/domains/example">Learn more</a></div></body></ht
ml>\n'
```

Another way of importing submodules is to use `from` - `import` statements. For example you can import `requests` submodule of `urllib` as:

In [10]:
```
from urllib import request
```

Then access the `request` module by simply `request` not `urllib.request`.

In [11]:
```
with request.urlopen("https://example.com/") as file:
  data = file.read()
```

Or you can import module objects directly like:

In [12]:
```
from urllib.request import urlopen
```

And use:

In [13]:
```
with urlopen("https://example.com/") as file:
  data = file.read()
```

In this method only the given functions, classes, objects are usable. To import everything from a module without using `.` when accessing use:

In [14]:
```
from urllib.request import *
```

Mathematical Functions

To use mathematical functions you need to import `math` module once.

In [15]:
```
from math import *
```

`math` module provides several mathematical functions. To use them write `math` then a `.` and the name of the function. Finally write the argument of that function in paranthesis.

Some of these functions are:

In [16]:
```
sqrt(16)
```

Out[16]:
```
4.0
```

In [17]:
```
sin(pi / 2)
```

Out[17]:
```
1.0
```

In [18]:
```
gcd(48, 180)
```

Out[18]:
```
12
```

In [19]:
```
factorial(5)
```

Out[19]:
```
120
```

Complex Functions

To use mathematical function on complex numbers import `cmath` module.

In [20]:
```
import cmath
```

Some of the functions defined in `cmath` module are:

In [21]:
```
cmath.sqrt(-16)
```

Out[21]:
```
4j
```

In [22]:
```
cmath.cos(3 + 2j)
```

Out[22]:
```
(-3.7245455049153224-0.5118225699873846j)
```

In [23]:
```
cmath.polar(3 + 4j)
```

Out[23]:
```
(5.0, 0.9272952180016122)
```

Logging Error Messages

In [24]:
```
import logging
```

In [25]:
```
logging.basicConfig(
    filename="exceptions.log",
    level=logging.DEBUG,
    # level = logging.ERROR,
    force=True,
)
```

In [26]:
```
logging.getLogger().isEnabledFor(logging.DEBUG)
```

Out[26]:
```
True
```

In [27]:
```
try:
  a = []
  a[1]
except IndexError as err:
  logging.error(f"IndexError: {err}")
  # logging.warning(f"IndexError: {err}")
```

```
  # logging.info(f"IndexError: {err}")
  # logging.debug(f"IndexError: {err}")
```

In [28]:
```
logging.info("program started successfully")
```

In [29]:
```
logging.debug("some technical details" "and another")
```